PANDAS

Introduction to PANDAS

INFOMAX
COMPUTER ACADEMY

- PANDAS (PANel DAta) is a high-level data manipulation tool used for analysing data.
- It is very easy to import and export data using Pandas library which has a very rich set of functions.
- It is built on packages like NumPy and Matplotlib and gives us a single, convenient place to do most of our data analysis and visualisation work.
- Pandas has three important data structures, namely – Series, DataFrame and Panel to make the process of analysing data organised, effective and efficient.

Installing Pandas

- Installing Pandas is very similar to installing NumPy. To
- install Pandas from command line, we need to type in:
  - pip install pandas

Data Structure in Pandas

INFOMAX
COMPUTER ACADEMY

- A data structure is a collection of data values and operations that can be applied to that data.
- It enables efficient storage, retrieval and modification to the data.
- commonly used data structures in Pandas are:
  - **Series** : it is one dimensional structure storing element of homogeneous (same data type) mutable (which can be modified) data.
  - **Dataframe** : it is a two dimensional structure storing heterogeneous (multiple data type) mutable data.
  - **Panel** : it is a three-dimensional way od storing items.

Series

- A Series is a one-dimensional array containing a sequence of values of any data type (int, float, list, string, etc) which by default have numeric data labels starting from zero.
- The data label associated with a particular value is called its index.
- We can also assign values of other data types as index.
- We can imagine a Pandas Series as a column in a spreadsheet.

Index	Value
0	Arnab
1	Samridh
2	Ramit
3	Divyam
4	Kritika

Creation of Series

- There are different ways in which a series can be created in Pandas.
  - Creation of Series from Scalar Values
  - Creation of Series from NumPy Arrays
  - Creation of Series from Dictionary

Creation of Series from Scalar Values


```
import pandas as pd
sr=pd.Series([4,5,6,7])
print(sr)
```

```
========================= RES
0    4
1    5
2    6
3    7
dtype: int64
```

```
import pandas as pd
month=['JAN','FEB','MAR','APR','MAY','JUN']
days=[31,28,31,30,31,30]
sr=pd.Series(days,index=month)
print(sr)
```

```
JAN    31
FEB    28
MAR    31
APR    30
MAY    31
JUN    30
dtype: int64
```

Creation of Series from NumPy Arrays

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
import numpy as np
arr=np.array([5,6,7,8])
sr=pd.Series(arr)
print(sr)
```

```
0    5
1    6
2    7
3    8
dtype: int32
```

Creation of Series from Dictionary

```
import pandas as pd
dict1={'India': 'NewDelhi', 'UK':'London', 'Japan': 'Tokyo'}
sr=pd.Series(dict1)
print(sr)
```

```
India     NewDelhi
UK           London
Japan         Tokyo
dtype: object
```

Creating Series with range() and for loop


```
import pandas as pd
sr=pd.Series(range(1,15,3),index=[x for x in 'abcde'])
print(sr)
```

```
a     1
b     4
c     7
d    10
e    13
dtype: int64
```

Accessing Elements of a Series

- There are two common ways for accessing the elements of a series: Indexing and Slicing.
- Indexing
  - By Position (Indexing like arrays/lists)
  - By Label (if index is custom)
  - Using .iloc[] (integer-location based indexing)
  - Using .loc[] (label-based indexing)
  - Boolean Indexing
- Slicing
  - By position (iloc):
  - By labels (loc):
- Get First / Last N Elements

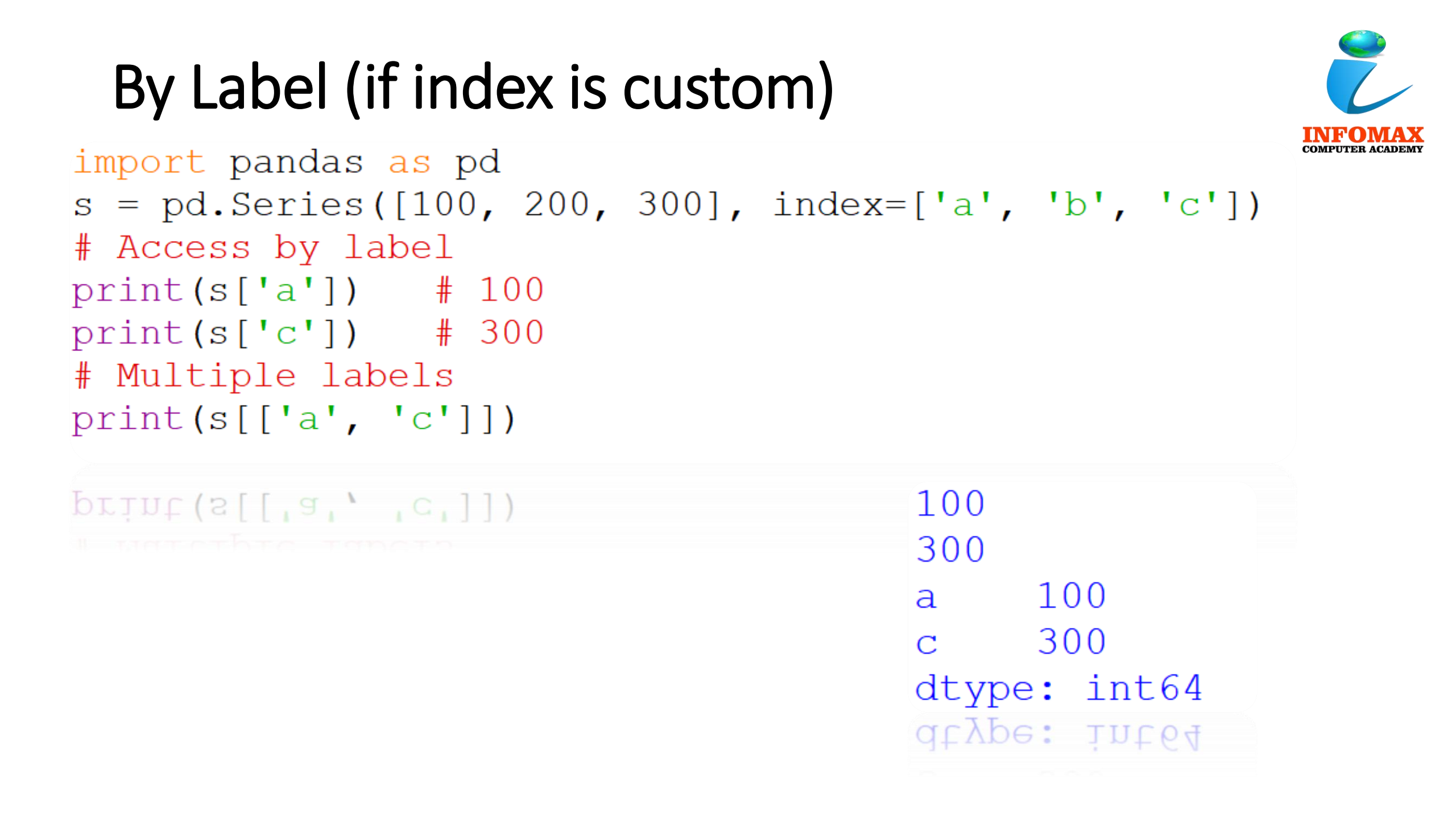
By Position (Indexing like arrays/lists)

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
s = pd.Series([10, 20, 30, 40, 50])
# Access single element
print(s[0])      # 10
print(s[3])      # 40
# Access multiple elements
print(s[[1, 3, 4]])
```

```
10
40
1    20
3    40
4    50
dtype: int64
```

By Label (if index is custom)




```
import pandas as pd
s = pd.Series([100, 200, 300], index=['a', 'b', 'c'])
# Access by label
print(s['a'])    # 100
print(s['c'])    # 300
# Multiple labels
print(s[['a', 'c']])
```

```
100
300
a    100
c    300
dtype: int64
```

Using .iloc[] (integer-location based indexing)

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
s = pd.Series([10, 20, 30, 40, 50])
print(s.iloc[0])      # 10
print(s.iloc[-1])     # 50
print(s.iloc[1:4])    # Slice -> 20, 30, 40
```

```
10
50
1    20
2    30
3    40
dtype: int64
```

Using .loc[] (label-based indexing)

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
s = pd.Series([100, 200, 300], index=['x', 'y', 'z'])
print(s.loc['x'])     # 100
print(s.loc[['x', 'z']])
```

```
100
x    100
z    300
dtype: int64
```

Boolean Indexing

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
s = pd.Series([5, 12, 18, 7, 25])
# Elements greater than 10
print(s[s > 10])
```

```
1    12
2    18
4    25
dtype: int64
```

Slicing By position (iloc):

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
s = pd.Series([10, 20, 30, 40, 50], index=['a','b','c','d','e'])
print(s[1:4])          # Default slicing (like Python list) → b, c, d
print(s.iloc[1:4])     # Position-based slice → same result
```

```
b    20
c    30
d    40
dtype: int64
b    20
c    30
d    40
dtype: int64
```

Slicing By labels (loc):

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
s = pd.Series([10, 20, 30, 40, 50], index=['a','b','c','d','e'])
print(s.loc['b':'d'])  # From 'b' to 'd' (inclusive!)
```

```
b    20
c    30
d    40
dtype: int64
```

Get First / Last N Elements


```
import pandas as pd
s = pd.Series([100, 200, 300, 400], index=['x', 'y', 'z', 'w'])
print(s.head(2))    # First 2 elements
print(s.tail(1))    # Last element
```

```
x    100
y    200
dtype: int64
w    400
dtype: int64
```

Attributes of Series

INFOMAX
COMPUTER ACADEMY

- Index : Shows the index (labels).
- Values : Returns the data as a NumPy array.
- Dtype : Data type of the elements.
- Shape : Shape of the Series (length,).
- Size : Total number of elements.
- Ndim : Number of dimensions (Series is always 1D).
- Name : Name of the Series (optional).
- is_unique : Checks if all values are unique.
- Hasnans : Checks if Series contains NaN
- Empty : True if Series is empty.

Example

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
import numpy as np
s = pd.Series([10, 20, 30, 20, np.nan], index=['a','b','c','d','e'])
s.name = "Marks"    # Assign a name
print("Series:\n", s, "\n")
print("Attributes of Series:")
print(" index      :", s.index)
print(" values     :", s.values)
print(" dtype      :", s.dtype)
print(" shape      :", s.shape)
print(" size       :", s.size)
print(" ndim       :", s.ndim)
print(" name       :", s.name)
print(" is_unique  :", s.is_unique)
print(" hasnans    :", s.hasnans)
print(" empty     :", s.empty)
```

Output

```
Series:
 a    10.0
b    20.0
c    30.0
d    20.0
e     NaN
Name: Marks, dtype: float64

Attributes of Series:
 index      : Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
 values     : [10. 20. 30. 20. nan]
 dtype      : float64
 shape      : (5,)
 size       : 5
 ndim       : 1
 name       : Marks
 is_unique  : False
 hasnans    : True
 empty     : False
```

Mathematical Operation on Series

INFOMAX
COMPUTER ACADEMY

- In **Pandas**, you can perform **mathematical operations on Series** just like you do with numbers or NumPy arrays. These operations are **element-wise** (applied to each value).

```
import pandas as pd
s1 = pd.Series([10, 20, 30, 40, 50])
s2 = pd.Series([5, 15, 25, 35, 45])
print("Series 1:")
print(s1)
print("\nSeries 2:")
print(s2)
print("\nAddition (s1 + s2):")
print(s1 + s2)
print("\nSubtraction (s1 - s2):")
print(s1 - s2)
print("\nMultiplication (s1 * s2):")
print(s1 * s2)
print("\nDivision (s1 / s2):")
print(s1 / s2)
print("\nModulus (s1 % s2):")
print(s1 % s2)
print("\nPower (s1 ** 2):")
print(s1 ** 2)
```

Vector Operations on Series

INFOMAX
COMPUTER ACADEMY

- In **Pandas**, a Series is essentially like a **vector** (1-D array) with labels. That means you can perform **vectorized operations** (element-wise) without writing loops.

```
import pandas as pd
s1 = pd.Series([10, 20, 30, 40, 50])
print("Series :")
print(s1)
print("\nAddition of 2 in each element")
print(s1 + 2)
print("\nprint only those element which is > 25")
print(s1>25)
print("\nexponential:")
print(s1 **2)
```

Retrieving value using condition

INFOMAX
COMPUTER ACADEMY

- We can also give condition to retrieve value from a series.

```
import pandas as pd
s = pd.Series([10, 25, 30, 45, 50, 75, 100])
print("Original Series:")
print(s)
print("\nValues greater than 30:")
print(s[s > 30])
print("\nValues <= 50:")
print(s[s <= 50])
print("\nValues between 20 and 60:")
print(s[(s >= 20) & (s <= 60)])
print("\nValues equal to 25 or 100:")
print(s[s.isin([25, 100])])
```

Deleting element from a series

INFOMAX
COMPUTER ACADEMY

- We can delete elements from series using **drop()** method by passing the index of the element to be deleted as the argument.

```
import pandas as pd
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
print("Original Series:")
print(s)
s1 = s.drop(2)#  Delete element by index Number
print("\nAfter deleting index 'c':")
s1 = s.drop('c')#  Delete element by index label
print("\nAfter deleting index 'c':")
print(s1)
s2 = s.drop(['b', 'd'])#  Delete multiple elements by index labels
print("\nAfter deleting indexes 'b' and 'd':")
print(s2)
```

Comparing the series

- In **Pandas**, you can compare two Series **element by element** using comparison operators (==, !=, <, >, <=, >=).

```
import pandas as pd
s1 = pd.Series([10, 20, 30, 40, 50])
s2 = pd.Series([10, 25, 20, 40, 60])
print("Series 1:")
print(s1)
print("\nSeries 2:")
print(s2)
print("\nEqual (s1 == s2):")
print(s1 == s2)
print("\nNot Equal (s1 != s2):")
print(s1 != s2)
print("\nGreater Than (s1 > s2):")
print(s1 > s2)
print("\nLess Than or Equal (s1 <= s2):")
print(s1 <= s2)
print("\nCommon values (s1[s1 == s2]):")
print(s1[s1 == s2])
```

Dataframe

INFOMAX
COMPUTER ACADEMY

- In **Pandas**, a **DataFrame** is the most important data structure. It is a **2-dimensional labeled data structure** (like an Excel sheet or SQL table) with rows and columns.
- We can create a dataframe using:
  - Dataframe from List
  - Dataframe from Series
  - Dataframe from Dictionary
  - Dataframe from NumPy ndarray

Creating Dataframe using List


```
import pandas as pd
data = [10, 20, 30, 40, 50]
df = pd.DataFrame(data)
print(df)
```

```
    0
0  10
1  20
2  30
3  40
4  50
```

```
import pandas as pd
data = [['Hindi', 30], ['English',56], ['maths',89]]
df = pd.DataFrame(data, columns=["subject", "marks"])
print(df)
```

```
   subject  marks
0    Hindi     30
1  English     56
2    maths     89
```

Creating Dataframe using Series

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
s = pd.Series([10, 20, 30, 40])
df = pd.DataFrame(s)
print(df)
```

```
    0
0  10
1  20
2  30
3  40
```

```
import pandas as pd
s1 = pd.Series([1, 2, 3], name="ID")
s2 = pd.Series(["Amit", "Sumit", "Rajesh"], name="Name")
df = pd.DataFrame({"ID": s1, "Name": s2})
print(df)
```

```
   ID    Name
0   1    Amit
1   2   Sumit
2   3  Rajesh
```

Creating Dataframe using Dictionary


```
import pandas as pd
data = {
    "ID": [1, 2, 3],
    "Name": ["Mohan", "Rohan", "Amit"],
    "Marks": [85, 90, 95]
}
df = pd.DataFrame(data)
print(df)
```

```
   ID   Name  Marks
0   1  Mohan     85
1   2  Rohan     90
2   3   Amit     95
```

Creating Dataframe using NumpPy Array

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
import numpy as np
arr = np.array([10, 20, 30, 40])
df = pd.DataFrame(arr)
print(df)
```

	0
0	10
1	20
2	30
3	40

```
import pandas as pd
import numpy as np
arr = np.array([[1, "Alice", 85],
                [2, "Bob", 90],
                [3, "Charlie", 95]])
df = pd.DataFrame(arr, columns=["ID", "Name", "Marks"])
print(df)
```

	ID	Name	Marks
0	1	Alice	85
1	2	Bob	90
2	3	Charlie	95

Attribute of Dataframe

INFOMAX
COMPUTER ACADEMY

Attribute	Meaning	Example Output
index	Labels of rows	RangeIndex(start=0, stop=3, step=1)
columns	Labels of columns	Index(['ID', 'Name', 'Marks'], dtype='object')
axes	Both row & column labels	[row_index, column_index]
dtypes	Data type of each column	int64, object
size	Total number of elements (rows × cols)	9
shape	Tuple → (rows, cols)	(3, 3)
ndim	Number of dimensions	2
empty	Checks if DataFrame is empty	False
T	Transpose → swap rows & columns	Rows ↔ Columns

Example

```
import pandas as pd
data = {
    "ID": [1, 2, 3],
    "Name": ["Amit", "Sumit", "Rahul"],
    "Marks": [85, 90, 95]
}
df = pd.DataFrame(data)
print("DataFrame:\n", df, "\n")
print("Index:", df.index)                    # Row index
print("Columns:", df.columns)                # Column labels
print("Axes:", df.axes)                      # Both row and column axis
print("Dtypes:\n", df.dtypes, "\n")          # Data types of columns
print("Size:", df.size)                      # Total elements (rows x cols)
print("Shape:", df.shape)                    # (rows, columns)
print("Ndim:", df.ndim)                      # Dimension of DataFrame
print("Is Empty?:", df.empty)                # Check if empty
print("Transpose:\n", df.T)                  # Transpose rows ↔ columns
```

Output


```
DataFrame:
    ID   Name  Marks
0    1   Amit     85
1    2  Sumit     90
2    3  Rahul     95

Index: RangeIndex(start=0, stop=3, step=1)
Columns: Index(['ID', 'Name', 'Marks'], dtype='object')
Axes: [RangeIndex(start=0, stop=3, step=1), Index(['ID', 'Name', 'Marks'], dtype='object')]
Dtypes:
 ID        int64
Name      object
Marks      int64
dtype: object

Size: 9
Shape: (3, 3)
Ndim: 2
Is Empty?: False
Transpose:
            0      1      2
ID          1      2      3
Name    Amit  Sumit  Rahul
Marks     85     90     95
```

Retrieving and accessing Rows/Column from datafarme

- 1. Accessing Columns

```
import pandas as pd
data = {
    "ID": [1, 2, 3],
    "Name": ["Alice", "Bob", "Charlie"],
    "Marks": [85, 90, 95]
}
df = pd.DataFrame(data)
print(df["Name"])# Single column (as Series)
print(df[["ID", "Marks"]]) # Multiple columns
```

```
0      Alice
1        Bob
2    Charlie
Name: Name, dtype: object
   ID  Marks
0   1     85
1   2     90
2   3     95
```

Retrieving and accessing Rows/Column from datafarme

- 2. Accessing Rows

```
import pandas as pd
data = {
    "ID": [1, 2, 3],
    "Name": ["Ram", "Mohan", "Sumit"],
    "Marks": [85, 90, 95]
}
df = pd.DataFrame(data)
#(a) By index using loc (label-based)
print(df.loc[0])          # Row with index label 0
print(df.loc[[0, 2]])     # Multiple rows
#(b) By position using iloc (integer-based)
print(df.iloc[1])         # Row at index position 1
print(df.iloc[[0, 2]])    # Multiple rows
#(c) Accessing Specific Cell
print(df.loc[0, "Name"]) # Ram
print(df.iloc[1, 2])           #  90
#(d)Accessing Rows with Condition
print(df[df["Marks"] > 85])
```

Adding/Modifying a row in a dataframe

INFOMAX
COMPUTER ACADEMY

- (a) Using loc[] with next index

```
import pandas as pd
df = pd.DataFrame({
    "ID": [1, 2],
    "Name": ["Alice", "Bob"],
    "Marks": [85, 90]
})
# Add new row at index 2
df.loc[2] = [3, "Charlie", 95]
print(df)
```

```
   ID     Name  Marks
0   1    Alice     85
1   2      Bob     90
2   3  Charlie     95
```

Adding/Modifying a row in a dataframe

INFOMAX
COMPUTER ACADEMY

- (b) Using append() (deprecated in new Pandas, use concat)

```
import pandas as pd
df = pd.DataFrame({
    "ID": [1, 2],
    "Name": ["Rajat", "pawan"],
    "Marks": [85, 90]
})
new_row = {"ID": 4, "Name": "David", "Marks": 88}
df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
print(df)
```

```
   ID   Name  Marks
0   1  Rajat     85
1   2  pawan     90
2   4  David     88
```

Adding/Modifying a row in a dataframe

- 2. Modifying an Existing Row

```
import pandas as pd
df = pd.DataFrame({
    "ID": [1, 2],
    "Name": ["Rajat", "pawan"],
    "Marks": [85, 90]
})
df.loc[1] = [2, "Komal", 92]
print(df)
```

```
   ID   Name  Marks
0   1  Rajat     85
1   2  Komal     92
```

Adding/Modifying a row in a dataframe

INFOMAX
COMPUTER ACADEMY

- (b) Modify specific cell in row

```
import pandas as pd
df = pd.DataFrame({
    "ID": [1, 2],
    "Name": ["Rajat", "pawan"],
    "Marks": [85, 90]
})
df.loc[0, "Marks"] = 95
print(df)
```

```
   ID   Name  Marks
0   1  Rajat     95
1   2  pawan     90
```

Renaming Column Name in Dataframe

```
import pandas as pd
df = pd.DataFrame({
    "ID": [1, 2,3],
    "Name": ["Rajat", "pawan","Shivam"],
    "Marks": [85, 90,86]
})
df = df.rename(columns={"Name": "StudentName", "Marks": "Score"})
print(df)
```

```
   ID StudentName  Score
0   1       Rajat     85
1   2       pawan     90
2   3      Shivam     86
```

Adding Column to a Dataframe

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
df = pd.DataFrame({
    "ID": [1, 2,3],
    "Name": ["Rajat", "pawan","Shivam"]
})
df["Marks"] = [85, 90, 95]
print(df)
```

```
   ID    Name  Marks
0   1   Rajat     85
1   2   pawan     90
2   3  Shivam     95
```

Deleting a Column from dataframe

INFOMAX
COMPUTER ACADEMY

- Using Drop() Function

```
import pandas as pd
df = pd.DataFrame({
    "ID": [1, 2,3],
    "Name": ["Rajat", "pawan","Shivam"],
    "Marks": [85, 90, 95]
})
df1 = df.drop("Marks", axis=1)    # axis=1 means column
print(df1)
```

```
   ID     Name
0   1    Rajat
1   2    pawan
2   3   Shivam
```

Deleting a Column from dataframe

INFOMAX
COMPUTER ACADEMY

- Using Del Statement

```
import pandas as pd
df = pd.DataFrame({
     "ID": [1, 2,3],
     "Name": ["Rajat", "pawan","Shivam"],
     "Marks": [85, 90, 95]
})
del df["Marks"]
print(df)
```

```
   ID    Name
0   1   Rajat
1   2   pawan
2   3  Shivam
```

Iterations in Dataframe

- <DFObject>.iterrows() : it represents dataframe row-wise, record by record.
- <DFObject>.iteritems() : it represents datafarme column-wise.

iterrows()

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
df = pd.DataFrame({
    "Name": ["Rajat", "pawan","Shivam"],
    "Marks": [85, 90, 95]
})
for index, row in df.iterrows():
    print(index, row["Name"], row["Marks"])
```

```
0 Rajat 85
1 pawan 90
2 Shivam 95
```

iteritems()


```
import pandas as pd
df = pd.DataFrame({
    "Name": ["Rajat", "pawan","Shivam"],
    "Marks": [85, 90, 95]
})
for label, content in df.items():
    print("Column:", label)
    print(content)
```

```
Column: Name
0      Rajat
1      pawan
2     Shivam
Name: Name, dtype: object
Column: Marks
0     85
1     90
2     95
Name: Marks, dtype: int64
```

Binary Operatotions

INFOMAX
COMPUTER ACADEMY

- Binary operations mean operations performed **between two DataFrames (or Series)**, element-wise.
  Examples: addition, subtraction, multiplication, division, and also logical operations.
- Common Binary Operations in DataFrames
  - Arithmetic Operations
  - Using Methods (add(), sub(), mul(), div())
  - Comparison Operators
  - Combine DataFrames with combine()

Arithmetic Operations


```
import pandas as pd
df1 = pd.DataFrame({"A":[1,2,3], "B":[4,5,6]})
df2 = pd.DataFrame({"A":[10,20,30], "B":[40,50,60]})
print("Addition \n",df1 + df2)
print("Subtraction \n", df1 - df2)
print("Multiplication \n",df1 * df2)
print("Division \n",df1 / df2)
```

```
Addition
    A   B
0  11  44
1  22  55
2  33  66
Subtraction
    A   B
0  -9 -36
1 -18 -45
2 -27 -54
Multiplicatior
    A    B
0  10  160
1  40  250
2  90  360
Division
     A    B
0  0.1  0.1
1  0.1  0.1
2  0.1  0.1
```

Using Methods (add(), sub(), mul(), div())


```
import pandas as pd
df1 = pd.DataFrame({"A":[10,20], "B":[30,40]})
df2 = pd.DataFrame({"A":[1,2], "B":[3,4]})
print("ADD \n",df1.add(df2))
print("SUB \n",df1.sub(df2))
print("MUL \n",df1.mul(df2))
print("DIV \n",df1.div(df2))
```

```
ADD
      A   B
0  11   33
1  22   44
SUB
      A   B
0   9  27
1  18  36
MUL
      A    B
0  10    90
1  40   160
DIV
         A      B
0  10.0  10.0
1  10.0  10.0
```

Comparison Operators


```
import pandas as pd
df1 = pd.DataFrame({"A":[10,20], "B":[30,40]})
df2 = pd.DataFrame({"A":[45,2], "B":[30,47]})
print(df1 > df2)
print(df1 == df2)
```

```
       A      B
0  False  False
1   True  False
       A      B
0  False   True
1  False  False
```

Combine DataFrames with concat()

INFOMAX
COMPUTER ACADEMY

- The concat() function in Pandas is used to **combine multiple DataFrames or Series** along rows or columns.
- **Syntax:**

```
pd.concat(objs, axis=0, join="outer", ignore_index=False)
```

  - **objs** → list/tuple of DataFrames or Series
  - **axis=0** → stack row-wise (default)
  - **axis=1** → stack column-wise
  - **join="outer"** → union of indexes (default)
  - **join="inner"** → intersection of indexes
  - **ignore_index=True** → reindex automatically

Example

```
import pandas as pd
df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})
result = pd.concat([df1, df2]) # Default axis=0
print(result)
result = pd.concat([df1, df2], axis=1)
print(result)
```

```
   A  B
0  1  3
1  2  4
0  5  7
1  6  8
   A  B  A  B
0  1  3  5  7
1  2  4  6  8
```

CSV FILE

INFOMAX
COMPUTER ACADEMY

- A **CSV file** (stands for **Comma-Separated Values**) is a simple text file used to store data in a **tabular format** (rows and columns).
- **Key points about CSV files:**
  - Each line in the file is a **row of data**.
  - Columns within a row are separated by a **comma** (,).
  - It’s often used to exchange data between applications like **Excel, Google Sheets, Databases, and Programming Languages**.
  - The file usually has the extension **.csv**.

Reading from a CSV file to Dataframe

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
df = pd.read_csv("data.csv")
print(df)
```

```
   Roll                Name      Mobile                          Email
0   101     FAKEHA KULSOOM  7784073066     saudnazish1990@gmail.com
1   102  MAHA IRFAN ULLAH   7992130953       mahairfan045@gmail.com
2   103    KULSOOM REHMAN   9795960246  kulsoomrehman2000@gmail.com
3   104    GRANDIN MAJOR V  9940243792       grandinmajor1@gmail.com
4   105     MOHD ASIM KHAN  9532963342  mohd.asimkhan9839@gmail.com
5   106       FAIQUA ARIF   7905584592     faiquaarif7006@gmail.com
```

Reading CSV file with specific/Selected Column

INFOMAX
COMPUTER ACADEMY

- When you don't want to load **all columns** from a CSV into your DataFrame, you can use the **usecols** parameter of pd.read_csv().

```
import pandas as pd
df = pd.read_csv("data.csv", usecols=["Name", "Mobile"])
print(df)
```

```
     Name      Mobile
0    Raju  7784073066
1  Pappu  7992130953
2    Sonu  9795960246
3    Monu  9940243792
4  Teepu  9532963342
```

Reading CSV file with Specific/Selected rows

- ‘nrows’ attribute used with read_csv() function to read specified number of lines from csv file.

```
import pandas as pd
# Read first 3 rows
df = pd.read_csv("data.csv", nrows=3)
print(df)
```

```
   Roll   Name      Mobile
0   101   Raju  7784073066
1   102  Pappu  7992130953
2   103   Sonu  9795960246
```

Skip rows while reading

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
# Skip first 2 rows
df = pd.read_csv("data.csv", skiprows=2)
print(df)
```

```
   102  Pappu  7992130953
0  103   Sonu  9795960246
1  104   Monu  9940243792
2  105  Teepu  9532963342
```

Reading CSV file without Header

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
df = pd.read_csv("data.csv", header=None)
print(df)
```

```
      0       1           2
0  Roll    Name        City
1   101   Rohit        Agra
2   102   Mohit      Kanpur
3   103   Sumit      Bhopal
4   104    Amit  Prayagraj
```

Reading SCV file without index

```
import pandas as pd
df = pd.read_csv("data.csv", index_col=0)
print(df)
```

```
        Name        City
Roll
101    Rohit        Agra
102    Mohit      Kanpur
103    Sumit      Bhopal
104     Amit   Prayagraj
```

Updating/Modifying Content in a csv File

- Import Module
- Open CSV file and read its data
- Find Column to be update
- Update value in the CSV file using to_csv() function.

Example

INFOMAX
COMPUTER ACADEMY

```
import pandas as pd
df = pd.read_csv("data.csv")
df.loc[1,'City']='Prayagraj'
df.to_csv("data.csv")
print(df)
```

```
   Roll   Name        City
0   101  Rohit        Agra
1   102  Mohit  Prayagraj
2   103  Sumit      Bhopal
3   104   Amit  Prayagraj
```

INFOMAX
COMPUTER ACADEMY